Screen Recorder — Native Plugin for Unity

Record your Unity game as H.264 MP4 — hardware-accelerated, zero performance hit.

Screen Recorder uses platform-native encoders (AVFoundation, Media Foundation) with async GPU readback so recording never blocks your main thread.

Features

- **Hardware H.264 encoding** — VideoToolbox (Apple), Media Foundation (Windows)
- **Two capture modes** — record the full game view or a specific camera
- **Async GPU readback** — frames are read off the GPU without stalling the render thread
- **Audio recording** — game audio, microphone, or both mixed
- **Overlay / watermark** — composite a logo or UI texture directly into the video
- **Configurable quality** — FPS (15–120), bitrate, quality preset, hardware or software encoder
- **IL2CPP compatible** — static native callbacks, no dynamic code generation
- **Simple API** — start/stop with two method calls; events for completion and errors

---

Platforms

Platform	Encoder	Min OS
macOS	AVFoundation + VideoToolbox	macOS 11+
Windows	Media Foundation	Windows 10

Package Contents

```
Assets/Draft/ScreenRecorder/
├── Runtime/
│   ├── ScreenRecorderBase.cs
│   ├── CameraRecorder.cs
│   ├── GameViewRecorder.cs
│   └── ScreenRecorderPlugin.cs
├── Editor/
│   ├── ScreenRecorderBaseEditor.cs
│   └── CameraRecorderEditor.cs
├── Plugins/
│   ├── macOS/  libScreenRecorder.dylib
│   └── Windows/ScreenRecorder.dll
├── Shaders/
│   ├── ScreenCaptureBlit.shader
│   └── RecordingComposite.shader
└── Demo/
    └── RecorderSample scene
```

Quick Start

1. Add `GameViewRecorder` or `CameraRecorder` to any GameObject in your scene.

2. Wire up events and call `StartRecording()` :

```
using ScreenRecorder;

public class RecordButton : MonoBehaviour
{
    [SerializeField] GameViewRecorder recorder;

    void Start()
    {
        recorder.OnRecordingComplete += path => Debug.Log("Saved: " + path);
        recorder.OnRecordingError   += err  => Debug.LogError(err);
    }

    public void Toggle()
    {
        if (recorder.IsRecording)
            recorder.StopRecording();
        else
            recorder.StartRecording();
    }
}
```

3. The output `.mp4` file is saved to the platform Videos/Movies folder automatically.

Components

GameViewRecorder

Captures the entire game view. Uses `ScreenCapture` + color-space blit shader to ensure correct colors in both Gamma and Linear projects.

CameraRecorder

Captures a specific `Camera`. Uses `OnRenderImage` so the frame is always correctly oriented — no platform-specific Y-flip issues.

Both components share the same inspector and events via the `ScreenRecorderBase` base class.

Inspector Settings

Setting	Description
Output Width / Height	Video resolution. Set to `0` to match source.
FPS	Target frame rate (15–120).
Quality	Encoder quality preset (0–10).
Bitrate	kbps. `0` = auto.

Use Hardware Encoder	Prefer GPU encoder when available.
Output Folder	Where to save `.mp4`. Empty = OS default.
Filename Prefix	Output: `{prefix}_{timestamp}.mp4`.
Enable Audio	Record game audio and/or microphone.
Enable Overlay	Composite a Texture2D watermark into the video.
Overlay Rect	Position and size in pixels.
Overlay Opacity	0–1.
Debug Mode	Verbose logging to Console.

Audio Recording

Enable audio and choose a source:

Source	Description
`None`	No audio track
`Game`	Game audio from `AudioListener`
`Microphone`	Device microphone input
`Both`	Mixed game + microphone

Audio capture attaches automatically to the scene's `AudioListener` at runtime and is removed when recording stops.

Overlay / Watermark

Assign a `Texture2D` to `overlayTexture` and set `overlayRect` (position + size in pixels). The alpha channel controls the blend shape.

```
recorder.enableOverlay  = true;
recorder.overlayTexture = logoTexture;
recorder.overlayRect    = new Rect(20, 20, 128, 128);  // top-left, 128×128
recorder.overlayOpacity = 0.8f;
```

Events

```
recorder.OnRecordingComplete += (string path)  => { /* file ready at path */ };
recorder.OnRecordingError    += (string error) => { /* handle error */ };
recorder.OnFrameCaptured     += (int count)    => { /* called each frame */ };
```

All events are dispatched on the **main thread** — safe to use with UI and Unity objects.

Runtime Status

```
recorder.IsRecording         // bool
recorder.State               // RecordingState: Idle / Recording / Stopping
recorder.FrameCount          // int — frames captured
recorder.ElapsedTime         // float — seconds
recorder.ElapsedTimeFormatted// string — "mm:ss"
recorder.RecordingFPS        // float — measured capture rate
recorder.CurrentOutputPath   // string — output file path
```

Hardware Encoding Check

```
bool hw = ScreenRecorderPlugin.IsHardwareEncodingAvailable();
string[] encoders = ScreenRecorderPlugin.GetAvailableEncodersArray();
```

Requirements

- Unity 2021.3 LTS or later
- Rendering: Built-in, URP, or HDRP
- IL2CPP and Mono both supported
- `AsyncGPUReadback` support required (all modern platforms)

ONLINE DOCUMENTATION

Documentation Hub

Support

For bug reports and questions: draft.sama@gmail.com

Full API documentation: API_REFERENCE.md